PID CONTROL

Understanding the controller through a spring–mass–damper example

Physical AI | Technical Assignment
Modern Monk Labs

In this report, I explain PID control and test it on the spring–mass–damper system specified in the assignment. I have kept the examples tied to the simulation so that the equations, graphs and Python code can be checked against one another.

Model: mass = 1 kg; damping = 10 N·s/m; spring stiffness = 20 N/m; target position = 1 m.

Implementation: Python with NumPy and Matplotlib. The PID calculation is written in the project code rather than delegated to a control library.

Candidate: Abhiram Baburaj

1. The basic idea

A controller is useful when a machine has to reach a target and stay there. It measures what the machine is doing, compares that measurement with the target, and changes the input based on the difference. The target is the **setpoint**, the measured value is the output, and target minus output is the **error**.

Think about setting the temperature in a shower. If the water is too cold, you turn up the hot tap. As it gets closer to the temperature you want, you ease off. If it becomes too hot, you turn the tap back down. You are using feedback: each adjustment depends on what happened after the previous one.

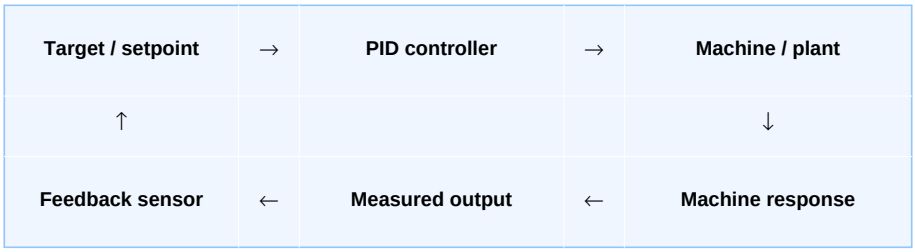


Figure 1. The sensor closes the loop by sending the measured output back to the controller.

PID follows this same idea, but it calculates the correction using three parts: the error now, the error accumulated over time, and how quickly the error is changing.

2. The PID formula

The continuous-time PID equation in parallel form is:

$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d de(t)/dt$

Here, $e(t)=r(t)-y(t)$. The reference r is the target and y is the measured output. In this example, error is in metres and the controller output is force in newtons. That gives K_p units of N/m, K_i units of N/(m·s), and K_d units of N·s/m.

Proportional (P): this part reacts to the error at the current instant. A larger K_p gives a stronger correction for the same position error. It often makes the response faster, but too much gain can lead to overshoot or oscillation.

Integral (I): this part adds up error over time. If the block remains a little short of the target, the integral keeps growing and increases the force. This is what can remove the steady-state offset. Too much integral action can make the response overshoot, and it can build up when the actuator is saturated.

Derivative (D): this part reacts to how quickly the error changes. It can oppose rapid movement and reduce bouncing. The trade-off is that differentiation makes measurement noise more noticeable.

In a computer, the controller runs at discrete time steps. With sample interval Δt, a basic version is $e[k]=r[k]-y[k]$, $I[k]=I[k-1]+e[k]\Delta t$, $D[k]=(e[k]-e[k-1])/\Delta t$, and $u[k]=K_p e[k]+K_i I[k]+K_d D[k]$. The submitted implementation filters the derivative and, by default, calculates it from the measurement rather than the setpoint error.

Using the negative measurement derivative for D avoids a sudden derivative spike when the setpoint changes. The P and I terms still use the setpoint error.

3. The test system

The assignment asks for a 1 kg block attached to a spring and a damper. The spring stiffness is 20 N/m and the damping coefficient is 10 N·s/m. Let x be the block position and u the force applied by the actuator.

Newton's second law gives **m x■ + c x■ + kx = u**. Substituting the assignment values gives **x■ = u** – **10x■** – **20x**, because m=1 kg.

The target is a step from 0 m to 1 m. Once the block has stopped at 1 m, its velocity and acceleration are both zero. The damper then contributes no force, but the spring still pulls back with 20×1 = **20 N**. The actuator must keep applying 20 N to hold the block at that position.

This explains an important feature of the experiment: a proportional controller needs some error to produce the holding force. Integral action can build up the required force even after the error becomes very small.

The simulation updates velocity from acceleration and then updates position using the new velocity (semi-implicit Euler integration). The normal runs use a 0.001 s time step.

4. How I judge the response

I use the four measures requested in the assignment:

Rise time: the time taken to move from 10% to 90% of the achieved final output. **Overshoot:** how far the peak rises above the 1 m target, expressed as a percentage. **Settling time:** the time after which the response stays inside a ±2% band around the target. **Final error:** the target minus the last simulated position.

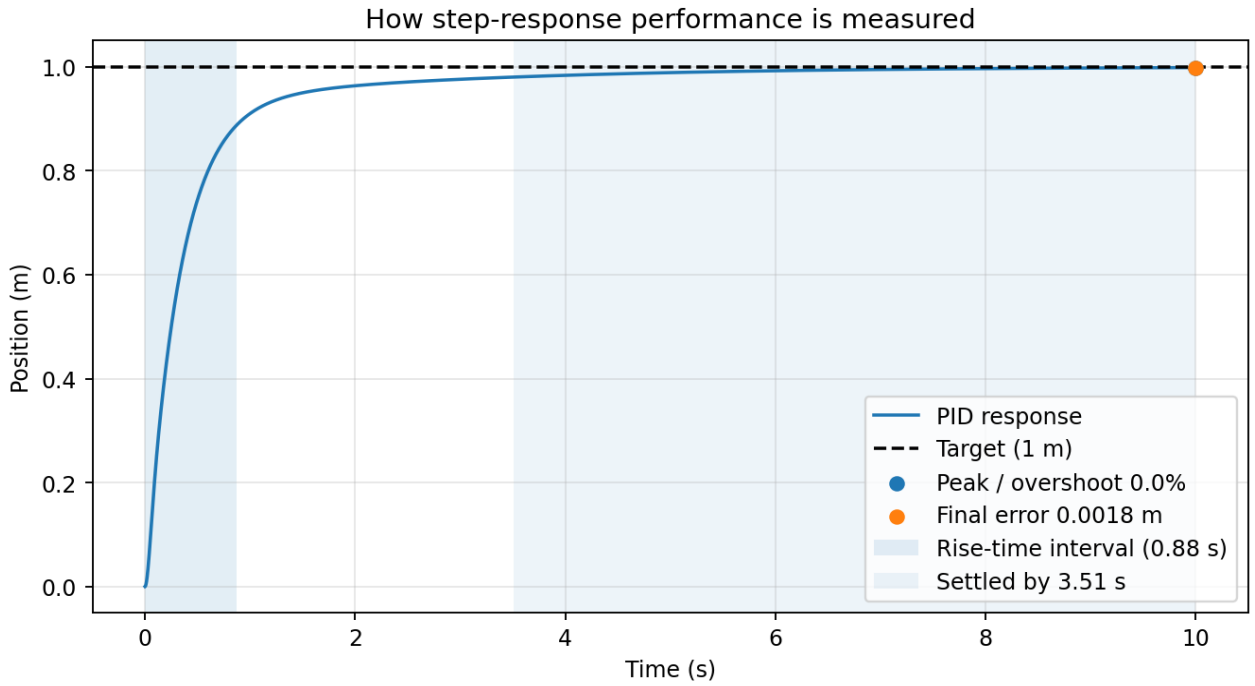


Figure 2. The graph marks the rise interval, peak, settling region and final sample for the PID response.

The code calculates these measures rather than estimating them by eye. If a response never stays within the ±2% band before the simulation ends, the table reports "Not settled".

5. Looking at P, then PI and PD

For P-only control, the final position can be found without running the simulation. At rest, the spring force and controller force balance: kx = Kp(r–x). Rearranging gives $x_\infty=[Kp/(Kp+k)]r$, so the final error is $e_\infty=[k/(Kp+k)]r$.

With k=20 N/m and r=1 m, the errors are 0.800 m for Kp=5, 0.500 m for Kp=20, 0.286 m for Kp=50 and 0.167 m for Kp=100. Increasing Kp reduces the error, but no finite P gain makes it exactly zero.

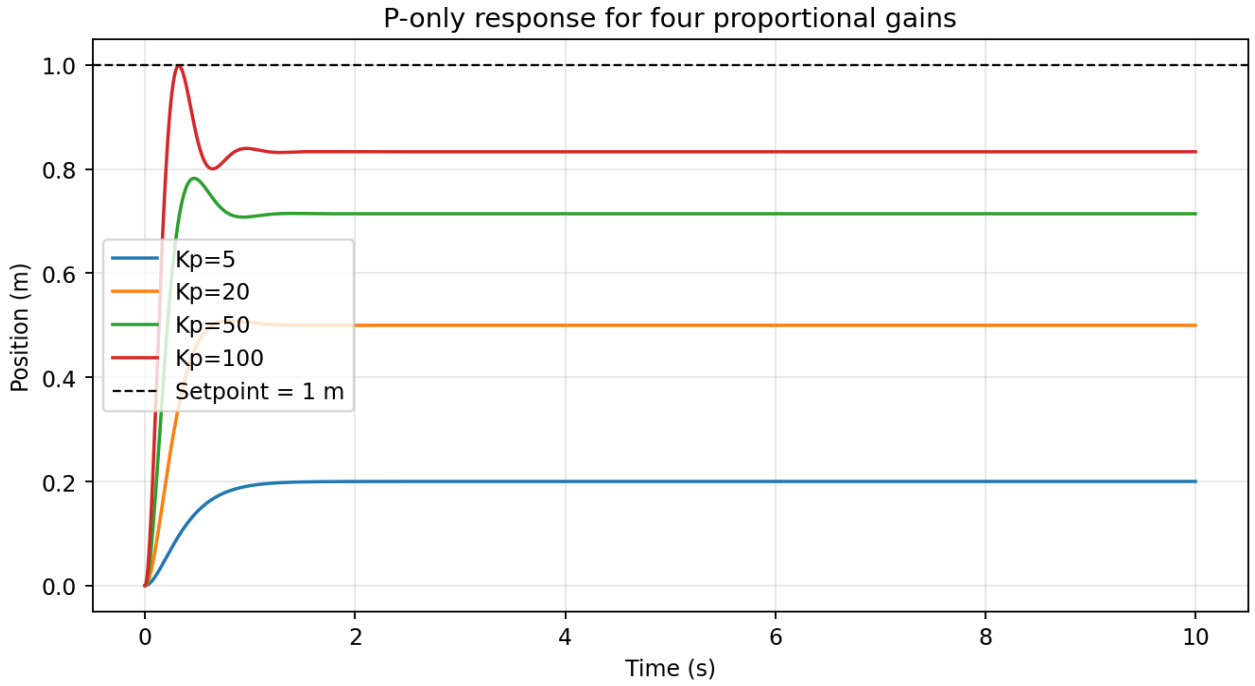


Figure 3. The higher-gain P responses finish closer to the target, but still have a steady-state offset.

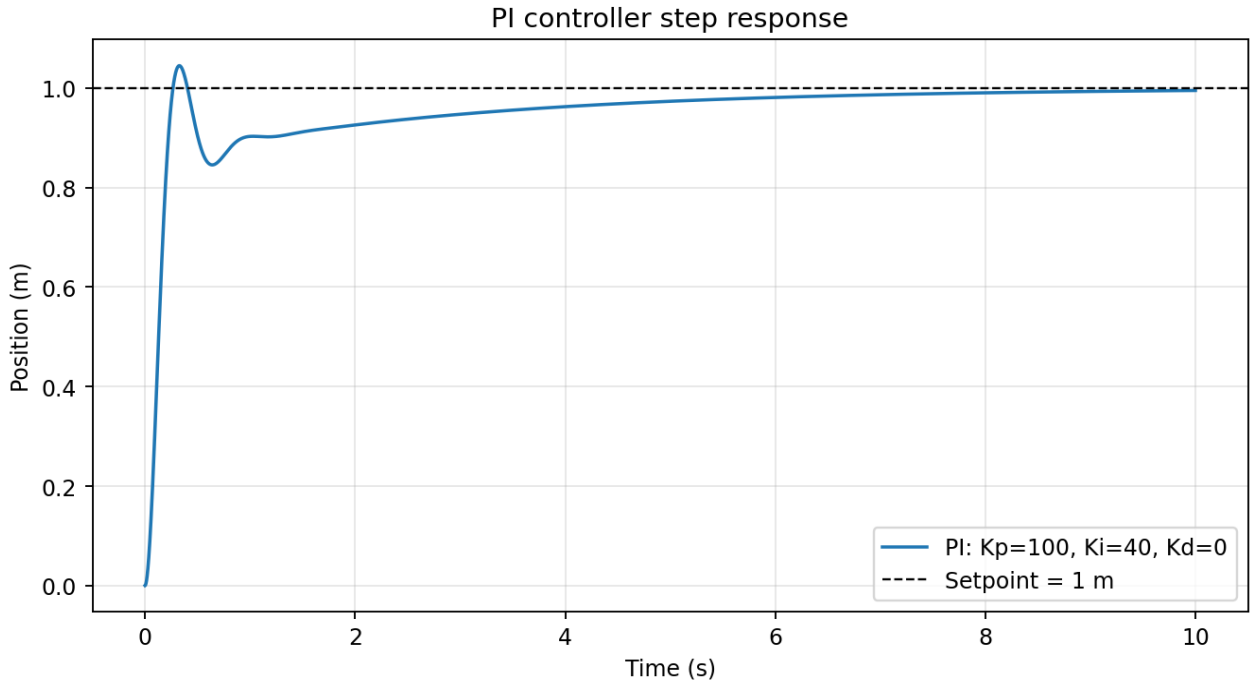


Figure 4. PI adds integral action to correct the remaining offset.

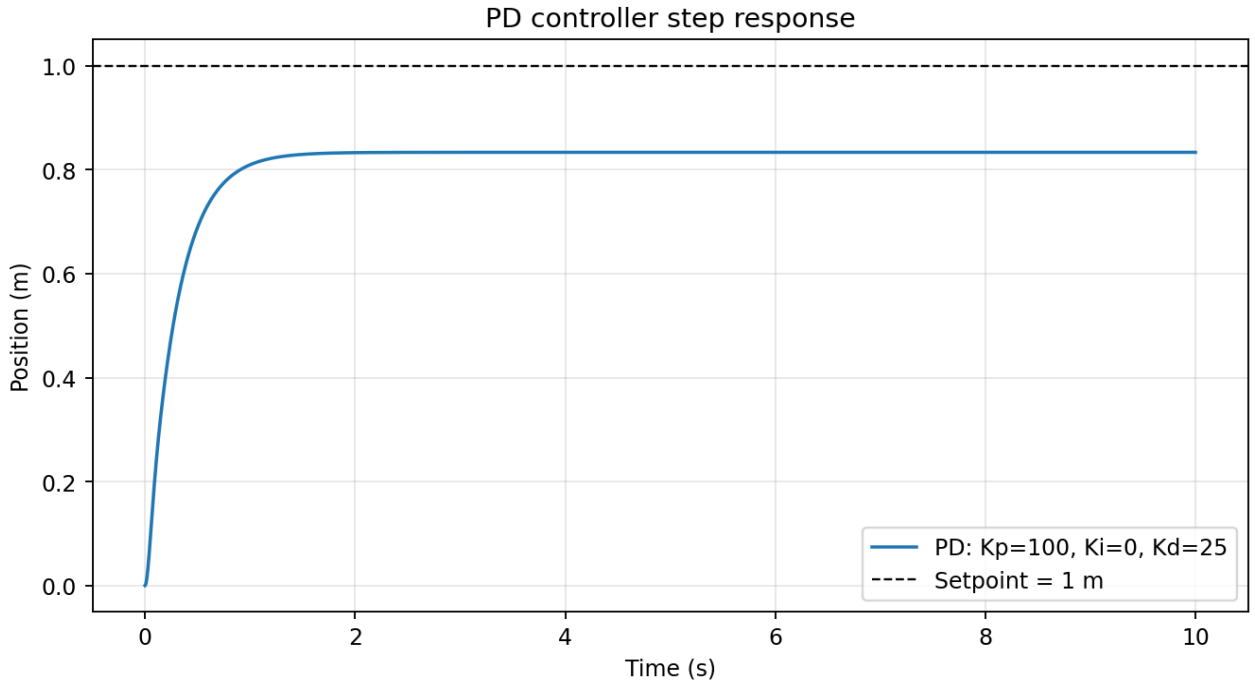


Figure 5. PD adds derivative action, which changes the transient response but does not remove the spring-load offset.

The integral term keeps adding the remaining error, so it can build the constant force needed to balance the spring. Derivative action changes how the block approaches the target; it does not provide the constant holding force by itself.

6. Comparing P, PI, PD and PID

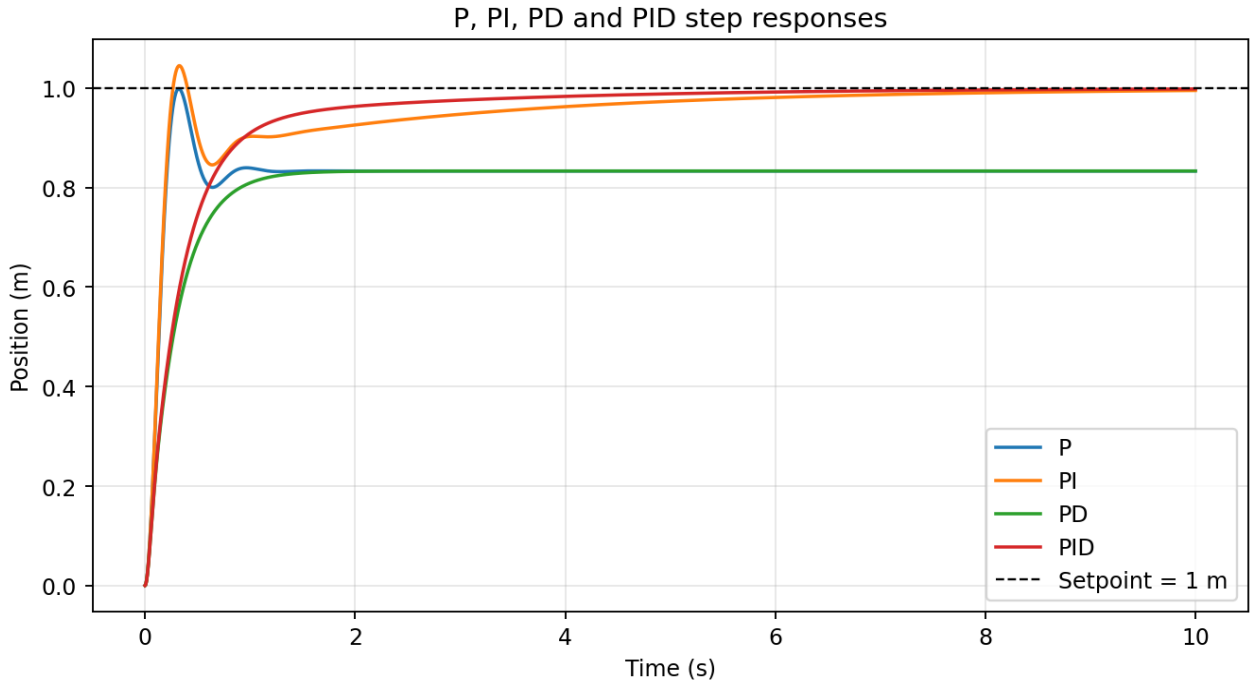


Figure 6. The four controller responses are shown together using the same plant and target.

Controller	Rise time (s)	Overshoot (%)	Settling time (s)	Final error (m)
P	0.142	0.000	Not settled	0.167
PI	0.172	4.510	5.820	0.005
PD	0.605	0.000	Not settled	0.167
PID	0.875	0.000	3.511	0.002

The gains used here are Kp=100, Ki=40 and Kd=25, with terms switched on or off for each controller. These are example values for this model, not settings that should be copied to every robot.

The P and PD cases do not settle inside the ±2% target band because both retain about 0.167 m of final error. PI and PID reduce that error to a few millimetres. PI has 4.51% overshoot in this run, while the chosen PID gains produce no overshoot and settle in about 3.51 s. In this example, the full PID response is not the fastest to reach 90% of its final value, but it gives a useful balance of accuracy and settling.

I would keep these gains as a starting point for this particular plant, then retune if the mass, spring stiffness, damping, actuator or sample time changed.

7. Problems that appear in real implementations

(a) Integral windup. The actuator in this test is limited to ±30 N. If the controller asks for more force, the output clips at the limit. Without protection, the integral can continue accumulating even though the actuator cannot provide the requested force. The code includes conditional integration: it pauses the integral when the output is saturated and the error would push it further into saturation.

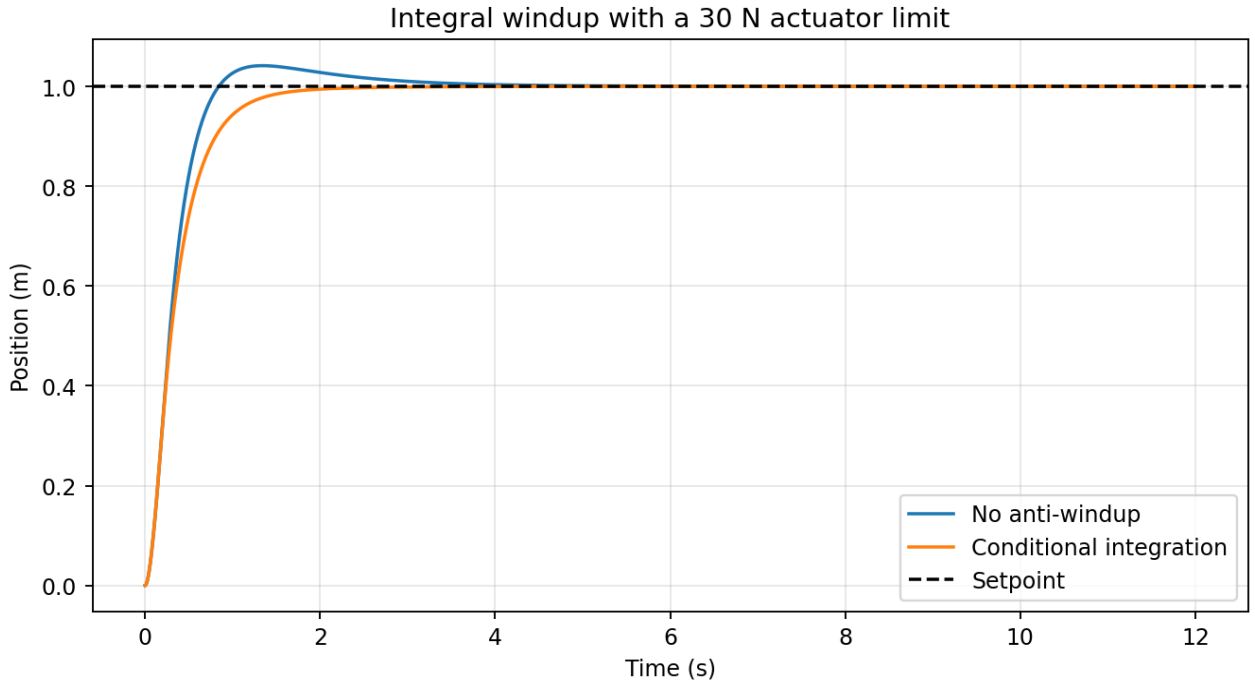


Figure 7. The force limit makes windup visible; conditional integration limits further accumulation.

(b) Sensor noise. The noise test adds a bounded ±1 mm disturbance to the measured position. A derivative term reacts strongly to rapid changes, including changes caused by noise. A low-pass filter on D reduces the resulting variation in control effort.

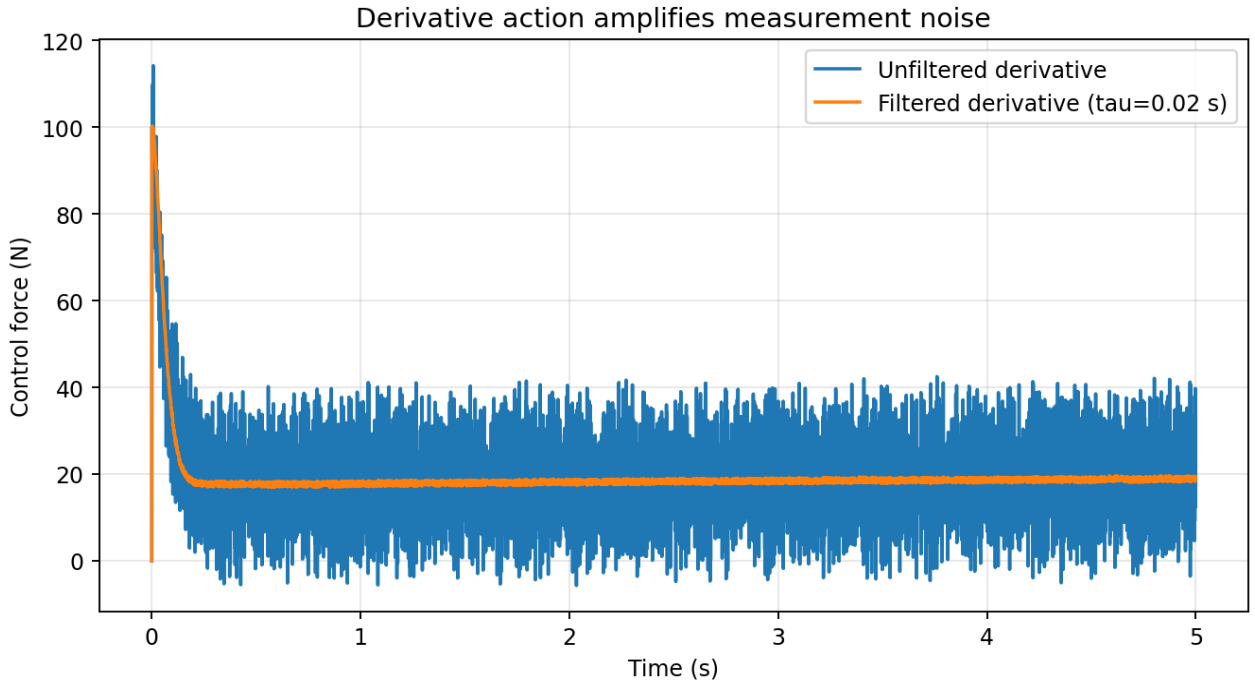


Figure 8. The derivative term is sensitive to noisy measurements; filtering reduces the high-frequency variation.

(c) Derivative kick. If D is calculated from error, a sudden setpoint change looks like a sudden error change and can create a large spike. Calculating D from the negative measured-output derivative avoids a spike caused by the setpoint alone.

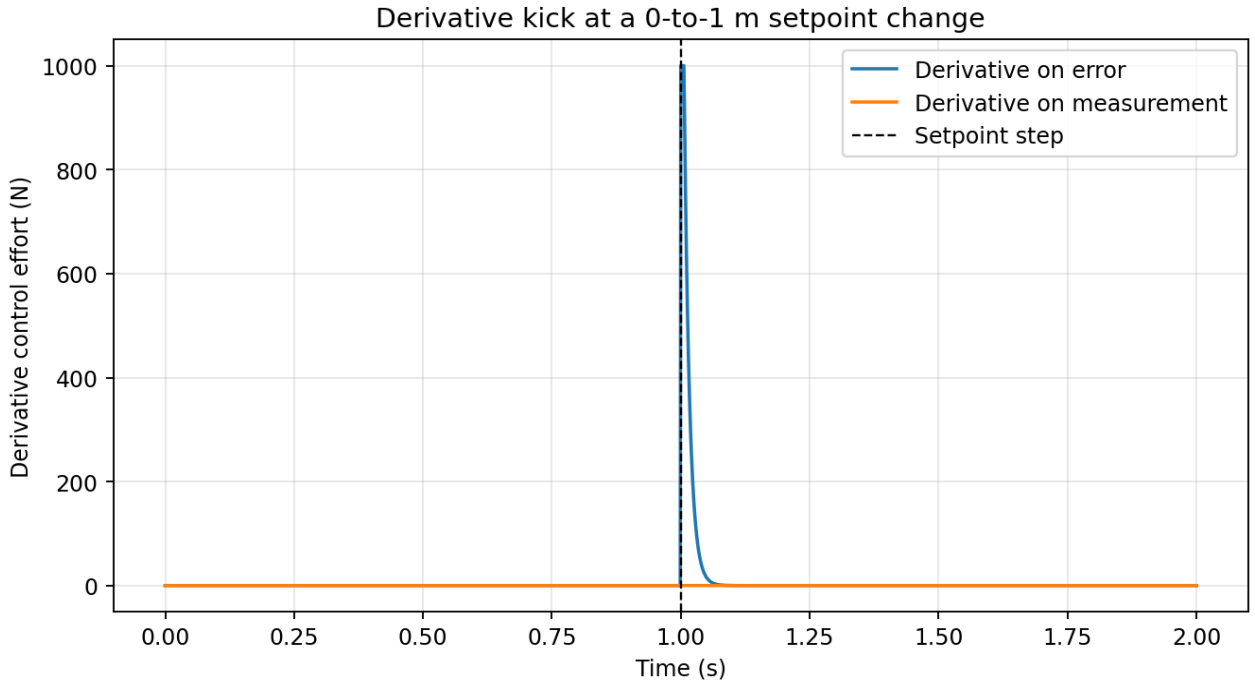


Figure 9. Derivative-on-measurement avoids the setpoint-induced kick.

(d) Slow updates. A controller that updates too slowly reacts to old information. It can miss important changes and may produce a poorer or unstable response. The update rate should be chosen with the plant dynamics and sensor/computation delays in mind.

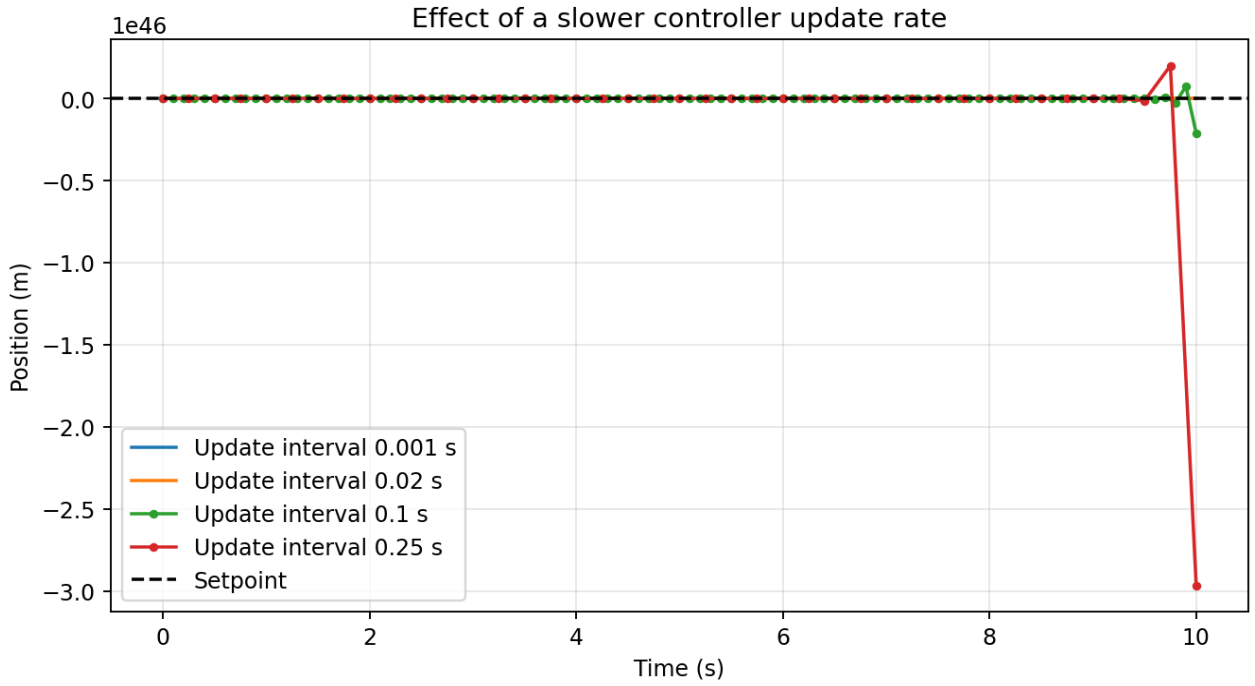


Figure 10. Coarser update intervals change the simulated step response.

8. Choosing the gains

For hand tuning, I would begin with Ki=0 and Kd=0. I would increase Kp in small steps and watch whether the response becomes fast without excessive overshoot or oscillation. Once P gives a reasonable transient response, I would add a small Ki to remove the remaining offset. I would then adjust Ki carefully and add Kd if the response needs more damping. After each change, I would check the graphs and the four performance measures.

The assignment also asks for Ziegler–Nichols tuning on $G(s)=1/(s+1)^3$. With proportional-only control, the characteristic equation is $(s+1)^3+Kp=0$. The Routh stability boundary gives $Ku=8$. At that boundary, the oscillation frequency is $\omega u=\sqrt{3}$ rad/s, so $Pu=2\pi/\sqrt{3}\approx 3.628$ s.

The classic PID recipe gives $Kp=0.6Ku=4.8$, $Ti=0.5Pu\approx 1.814$ s and $Td=0.125Pu\approx 0.453$ s. In parallel form, $Ki=Kp/Ti\approx 2.646$ and $Kd=KpTd\approx 2.177$. These gains are a starting point; the response still needs to be checked and refined.

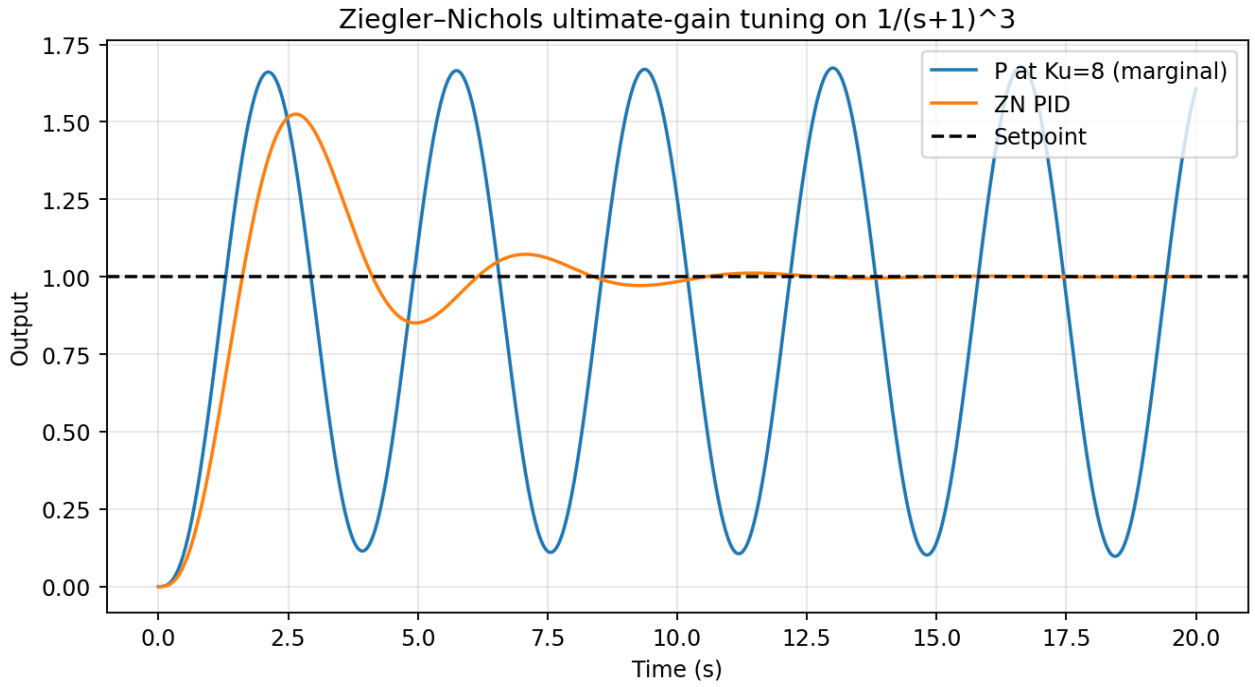


Figure 11. Ziegler–Nichols ultimate-gain tuning on the third-order test plant.

9. The Python implementation

The main file is `src/run_all.py`. It contains a reusable `PID` class and a separate spring–mass–damper simulator. The controller supports output limits, conditional-integration anti-windup, derivative filtering and derivative-on-measurement. Keeping the controller separate from the plant makes it easier to test the same controller structure on another model.

The script uses NumPy for the numerical calculations and Matplotlib for the figures. It does not use a ready-made PID or control library. A fixed random seed is used so that the sensor-noise plots can be recreated.

From the repository root, install the packages with `pip install -r requirements.txt` and run `python src/run_all.py`. The graphs are saved in the `figures/` folder, and the script prints the response metrics and the Ziegler–Nichols gains.

10. Real robots and common PID myths

Motor speed: an encoder measures shaft speed and the controller adjusts the motor command to follow a requested RPM, even when the load changes. **Drone attitude:** gyroscope and accelerometer measurements help estimate orientation; PID loops adjust motor thrust to stabilize roll, pitch and yaw. **Robot arm:** joint encoders measure angle and the controller drives each joint toward its commanded position. **Line follower:** optical sensors estimate how far the robot has moved away from the line, and PID adjusts the wheel speeds to bring it back.

There are a few assumptions about PID that are worth correcting:

Myth 1: Higher gains are always better. Higher gains can make a response faster, but too much gain can cause overshoot, oscillation or instability.

Myth 2: PID always removes every error. Integral action can remove steady-state error in a suitable stable system, but actuator saturation and other practical limits can prevent that.

Myth 3: Derivative always improves the response. Derivative can add damping, but it can also amplify sensor noise.

Myth 4: One set of gains works for every robot. Gains depend on the plant, payload, actuator, sensors and update rate. A change in any of these may require retuning.

Reproducing the work

The project includes the source code, dependency list, README, this report and the generated figures. To reproduce the work, create a Python environment, install the packages in `requirements.txt`, and run `python src/run_all.py` from the project root.

AI-use note: I used an AI tool for help with outlining and language review. I checked the technical content against the assignment and included the simulation code so the calculations and plots can be reproduced. I am responsible for reviewing the final work and being able to explain the implementation.

Limitations: this is an ideal linear simulation. It does not fully model motor electrical dynamics, backlash, friction changes, actuator rate limits or real sensor delays. Those would be useful additions before applying the controller to hardware.